Mon premier labyrinthe (édition Junior) : guide de l'enseignant

En bref

Âge	8 à 9 ans, avec l'édition **Junior** de PyGameMaker
Durée	90 minutes
Les élèves créent	Un labyrinthe sur un seul écran : dessiner un héros et un but, déplacer le héros avec les flèches, s'arrêter devant les murs, gagner au but, exporter en un seul fichier .html
Il faut	PyGameMaker en mode Junior, le projet de départ, un navigateur web. Internet n'est **pas** nécessaire pour jouer à ce jeu
Fiche de l'élève	Labyrinthe Junior : fiche de l'élève (une par élève)

Toute la leçon utilise uniquement des blocs qui existent dans la boîte à outils Junior. Le jeu terminé a été lancé dans le vrai moteur de jeu, exporté, puis joué dans un navigateur pour le vérifier.

Avant la leçon (15 minutes, une seule fois)

1. **Active le mode Junior** sur chaque ordinateur : Outils > Préférences > Général > Édition de l'IDE > Junior, puis redémarre l'IDE.
2. **Copie le projet de départ** (Junior-Maze-Starter.zip, sur la page du wiki ou dans wiki/downloads/) et décompresse-le pour que chaque élève ait un dossier appelé **My Maze**.
3. **Essaie la leçon toi-même une fois.** La version terminée se trouve dans Junior-Maze-Finished.zip, pour comparer.
4. **Décide comment le jeu arrivera sur un téléphone** (voir « Jouer sur un téléphone » plus bas). Fais-le avant le cours, pas après.
5. Imprime la fiche de l'élève, ou affiche-la sur un deuxième écran.

> si un ordinateur a déjà été réglé sur un autre thème ou une autre police dans les Préférences, le texte plus grand de Junior ne s'appliquera pas. Réinitialise-les à cet endroit.

Ce que contient le projet de départ

Une image de mur marron (spr_wall), un objet mur **Solide** (obj_wall) et une salle vide de 640 sur 416 (room0) dont le contour est déjà construit. Les élèves dessinent eux-mêmes leur héros et leur but, et font le reste. La salle est sur une grille de 32 pixels et l'aimantation à la grille est activée.

Taille de la fenêtre. La fenêtre du jeu a exactement la taille de la salle, volontairement petite ici (640 sur 416 pixels, soit 20 sur 13 carrés) pour qu'un labyrinthe entier puisse être construit et vérifié en une leçon. Si un élève veut une zone de jeu plus grande, n'étire pas la salle (le contour resterait ouvert sur le nouveau bord) : propose-lui la leçon suivante, « Labyrinthe au trésor », dont la salle de départ fait 960 sur 640 pixels.

Plan de la leçon (90 minutes)

Temps	Partie	Ce qui se passe	Si tu es en retard
0-10	Échauffement	Montre le jeu terminé (joue-y, idéalement sur un téléphone). Demande : de quoi a-t-il besoin ? Un héros, des murs, un but	Saute la discussion
10-15	1. Ouvrir	Ouvrir le projet de départ, appuyer sur F5	
15-30	2. Sprites	Créer un sprite deux fois (héros, but)	Donne les images du projet terminé
30-50	3. Héros	Créer un objet, six lignes de Blockly	Montre la première ligne au tableau, ils la copient
50-60	4. But	Un objet, deux blocs	
60-80	5. Labyrinthe	Peindre les murs, placer le héros et le but, tester	Donne un labyrinthe prêt : ouvre la salle du projet terminé
80-88	6. Export	Exporter en HTML5, ouvrir dans le navigateur	Fais l'export en démonstration devant la classe
88-90	Conclusion	Échanger et jouer ; les questions « Réfléchis »	

Les parties 2 à 4 sont les mêmes pour tout le monde. La partie 5 est celle où les élèves sont créatifs : protège ce temps.

Déroulement et problèmes courants

Partie 2, le dessin. Créer un sprite ouvre une image vide de 32 sur 32 dans l'éditeur de sprites. Rappelle aux élèves de cliquer sur Enregistrer dans l'éditeur. Un nom de sprite déjà utilisé donne un message « Name Already Used » (affiché en anglais pour l'instant) au lieu d'écraser l'ancien.

Partie 3, le héros. Six événements : quatre flèches maintenues, Aucune touche, et une collision avec obj_wall. Les lignes se copient : clic droit > Dupliquer, puis on change la touche et le nombre.

- Sans **Aucune touche**, le héros ne s'arrête jamais.
- Sans la ligne de **collision**, le héros traverse les murs : le fait que le mur soit Solide ne suffit pas à l'arrêter (vérifié : il est sorti de la salle). La ligne de collision avec **arrêter le mouvement** est ce qui fait d'un mur un vrai mur.
- Une vitesse de 4 déplace le héros de 4 pixels à chaque pas, 60 pas par seconde.

Partie 4, le but. C'est l'objet but qui contient la logique de victoire : quand collision avec obj_hero : Afficher message, puis Redémarrer le niveau. Le message peut dire ce qu'on veut, la ponctuation ne pose pas de problème.

Partie 5, le labyrinthe.

- **Deux carrés de large.** C'est la règle la plus importante. Le héros s'arrête là où on relâche la touche, pas sur la grille. Dans un chemin d'un carré, il peut se retrouver décalé de quelques pixels et rester coincé. Tous les couloirs du labyrinthe de référence font deux carrés de large, et un test le joue avec le héros placé jusqu'à 24 pixels hors de la grille : il atteint toujours le but.
- Garder la souris enfoncée en glissant peint une rangée de murs. Un clic en place un seul.
- Vérifie que le labyrinthe peut être terminé avant l'échange.
- Place le héros et le but à l'intérieur du contour, sur des carrés vides.

Exporter

Fichier > Exporter en HTML5 demande un dossier et propose d'ouvrir le jeu dans le navigateur. Le résultat est **un seul fichier**, nommé d'après le projet. Ce jeu ne contient aucun code Python : il n'a pas besoin d'internet pour jouer. La barre au-dessus du jeu affiche **Score** et **Lives** ; ce labyrinthe ne les utilise pas, donc les élèves peuvent poser des questions. On peut les ignorer.

Jouer sur un téléphone

Ce qu'on sait :

- La page exportée détecte un écran tactile et affiche alors quatre boutons flèches sous le jeu. Garder un bouton appuyé déplace le héros. Cela a été vérifié dans un navigateur de la taille d'un téléphone, sur un ordinateur, et **pas sur un vrai téléphone**.
- La page du jeu est faite pour les écrans étroits et se réduit pour s'y adapter.

Ce qu'il faut prévoir :

- **Faire arriver le fichier sur le téléphone.** Les téléphones ne peuvent souvent pas ouvrir un fichier .html depuis leur stockage (les iPhone en particulier). La méthode fiable est de mettre le fichier à une **adresse web** (l'espace web de l'école ou un hébergeur gratuit), puis d'ouvrir cette adresse sur le téléphone. Cela n'a pas été testé ici.
- **Vitesse sur un écran rapide.** Le jeu exporté fait un pas à chaque rafraîchissement de l'écran. Sur un écran de 60 Hz, c'est la vitesse prévue. Sur un téléphone avec un écran de 90 ou 120 Hz, le héros ira plus vite, jusqu'à deux fois plus vite. C'est une limite connue de l'export HTML5, pas encore corrigée. Un labyrinthe reste jouable, mais pense-y pour tout jeu qui demande un timing précis.

Différenciation

- **Besoin d'aide :** pars de la salle du projet terminé et ne change que les murs, ou donne un labyrinthe imprimé à recopier.
- **Pour aller plus loin :** ajoute des pièces qui donnent des points (le tutoriel 2, « Votre premier jeu », enseigne les blocs de score), ou un deuxième but. Les deux utilisent des blocs présents dans la boîte à outils Junior.

Évaluer le travail

Demande à chaque élève de te montrer : un sprite qu'il a dessiné, un héros qui bouge et s'arrête devant les murs, un labyrinthe qui peut être terminé, et le fichier .html exporté qui fonctionne dans un navigateur. Les trois questions « Réfléchis » servent de ticket de sortie.

Dépannage

Symptôme	Cause et solution
Le héros ne bouge pas	Le héros n'est pas placé dans la salle, ou il manque les lignes des flèches

Le héros traverse les murs	Il manque la ligne de collision (**Quand collision avec** obj_wall, **arrêter le mouvement**) sur le héros
Le héros est coincé dans un chemin	Le chemin fait un carré de large. Élargis-le
Rien au but	Le but a besoin de l'événement de collision avec obj_hero
« You win! » se répète	Il doit s'afficher une fois, puis la salle redémarre. Vérifie que **Redémarrer le niveau** est sous le même événement
Fenêtre de jeu grise	Ordinateur lent ou à court de mémoire. Attends, puis réessaie
Quelque chose a été supprimé	Outils > Restaurer les ressources supprimées